RIVET

Pivotal Insights | Focused Solutions

ALMS - API - Test Plan

March 31, 2026

RIVET
Pivotal Insights | Focused Solutions

Table of Contents

1. Purpose 3
2. Version Control 4
3. Terms and Definitions 5
4. Introduction 7
5. Technologies Used 8
6. Scope 8
7. Test Categories 9
8. Assumptions & Constraints 10
9. API Testing Approach 11
10. Validation phase deliverables 13
11. Raising a defect/logging a ticket 14
12. Testing phase entry/exit criteria 15
13. Validation phase / Test schedule 16
14. QA team responsibilities 17
15. Risks & Contingencies 18
16. Approvals 19

1. Purpose

- Define the testing deliverables of the project.
- Communicate to all responsible parties the Test Strategy.
- Define the roles and responsibilities of the QA Team members.
- Communicate to all responsible parties the identified Risks & Contingencies.

2. Version Control

Date	Version	Author	Description of Changes
31/03/2026	v1.0	Mahek Patel	Initial Draft

3. Terms and Definitions

Term/Acronym	Definition
Acceptance criteria	The exit criteria that a component or system must satisfy to be accepted by a user, customer, or other authorized entity
Blocked test case	A test case that cannot be executed because the preconditions for its execution are not fulfilled, or a defect is encountered during test execution, due to which further testing cannot be performed
Component	A minimal software item that can be tested in isolation
Conversion Testing	Testing of software used to convert data from existing systems for use in replacement systems
Defect	A flaw in a component or system that can cause the component or system to fail to perform its required function, e.g., an incorrect statement or data definition. A defect, if encountered during execution, may cause a failure of the component or system
Defect Management	The process of recognizing, investigating, taking action, and disposing of defects. It involves recording defects, classifying them, and identifying the impact
Expected Result	The behavior predicted by the specification, or another source, of the component or system under specified conditions.
Actual Result	The behavior produced/observed when a component or system is tested
Fail	A test is deemed to fail if its actual result does not match its expected result
Functional testing	Testing based on an analysis of the specification of the functionality of a component or system without reference to the internal structure of the component or system
Pass	A test is deemed to pass if its actual result matches its expected result
Precondition	Environmental and state conditions that must be fulfilled before the component or system can be executed with a particular test or test procedure
Regression testing	Testing of a previously tested program following modification to ensure that defects have not been introduced or uncovered

	in unchanged areas of the software, as a result of the changes made.
Test script	Commonly used to refer to the instructions for a particular test that will be carried out by an automated test tool.

4. Introduction

- The Asset & License Management System is a backend-driven enterprise application designed to manage and track company hardware assets such as laptops, desktops, servers, and accessories, along with software licenses assigned to employees throughout their lifecycle. The system centralizes asset inventory, ownership history, assignment details, and license utilization data to improve operational efficiency and ensure audit readiness.

- The application supports two roles: Employee and IT Admin. Employees can raise requests for assets or licenses and view their assigned resources, while IT Admin users are responsible for managing assets, licenses, approvals, assignments, returns, and inventory through secure role-based access.

- It provides end-to-end tracking for key operations, including asset creation, categorization, assignment, return management, status tracking (available, assigned, damaged, under repair), and prevention of invalid re-assignment. In license management, the system supports allocation to employees or assets, lifecycle tracking, historical usage visibility, and configurable expiry reminders (30/15/7 days).

- To ensure governance and traceability, the system maintains detailed audit logs for all critical actions such as create, update, assign, and delete operations. It also provides dashboard and reporting APIs that give insights into asset distribution, license status, inventory levels, and audit records. These capabilities help IT teams reduce manual effort, maintain data accuracy, and proactively manage risks related to asset usage and license expiry.

5. Technologies Used

Development Tools

Visual Studio Code (Version 1.113.0)

MS SQL Server (Version 19.3.4.0)

Framework & Libraries

.NET 8 (ASP.NET Core).

NuGet Package Manager:

Microsoft Entity Framework Core (8.0.0)

Microsoft Extensions Configuration (8.0.0)

Dependency Injection (DI) (8.0.0)

Swashbuckle.AspNetCore (6.6.2)

Microsoft Entity Framework Core SQL Server (8.0.0)

Microsoft.AspNetCore.OpenApi (8.0. 22)

Development Language

Visual C# .NET 8

Operating System:

Microsoft Windows 10 Pro

Communication

RESTful API (HTTP/HTTPS)

Supports:

GET

POST

PUT

DELETE

Data Format:

JSON

Authentication

JWT (JSON Web Token - Bearer)

Enables role-based login (Employee, IT Admin)

Storage:

SQL Server Management Studio 2024

7. Version Control

GitLab

8. Documentation & Testing

Swagger (API Documentation)

Postman

6. Scope

Scope:

- IT Admin: the primary operational user of the system. This role should be described as the person who logs in, creates and updates assets and licenses, assigns and returns assets, approves or rejects requests, changes statuses, reviews inventory, generates reports, exports data, and monitors audit or dashboard information. Use this role for scenarios that require full control, administrative permissions, and system-wide visibility.

- Employee: the restricted self-service user of the system. This role should be described as the person who logs in to view only permitted information, raise requests, check assigned assets or licenses, receive notifications, and perform only actions explicitly allowed by policy. Use this role for scenarios that focus on limited access, request submission, visibility of assigned items, and permission-based behavior.

7. Test Categories

Test Categories	In Scope?	Comments/Explanations
API	Yes ☒ No ☐	Validate API endpoints (task CRUD, approval workflow, reporting, configuration endpoints) for correctness, performance, security, and data handling. Core focus of this plan.
Access & Authentication	Yes ☒ No ☐	Login and authorization verification of authorized users. Validates role-based access control (Employee, IT Admin) and ensures proper RBAC enforcement across all endpoints.

8. Assumptions & Constraints

Assumptions

- QA environments are stable, reachable, and have required network/API access.
- API specifications (endpoints, schemas, status codes, error models) are baselined before test execution.
- Test users for all roles (Employee, IT Admin, and any approver roles) are pre-provisioned with valid credentials/tokens and appropriate permissions.
- Required seed data for assets, licenses, allocations, and request workflows is preloaded and refreshed as needed for repeatable testing.
- Necessary test data for assets, licenses, and request workflows will be pre-configured, and all external dependencies, such as email services, background job schedulers, and database access, will be operational during testing.
- External dependencies (mail service, reporting jobs, DB access) are available during planned test windows.

Constraints

- Mid-cycle API contract changes can cause frequent test case/script rework and impact execution timelines.
- Limited production-like data volume and quality may reduce confidence in reporting accuracy and performance realism.
- Time-dependent workflows (reminders, renewals, expirations, and month-end reports) require controllable system time or manual trigger support.
- Dependency instability or outages (email, scheduler, DB, auth, third-party services) can block end-to-end validation.
- Restricted direct DB/log access may limit deep state verification for some backend-only flows.

9. API Testing Approach

Below are the API testing approaches:

1. **HTTP status code validation**

   Verify that each API returns the correct status code for the scenario, such as 201 Created for successful creation, 200 OK for successful retrieval or update, 400 Bad Request for invalid input, 401 Unauthorized for missing or invalid credentials, 403 Forbidden for unauthorized role access, 404 Not Found for invalid resource IDs, and 409 Conflict for duplicate or conflicting requests.

2. **Response payload validation**

   Validate the response body for correct JSON structure, field names, data types, required values, and business rules. Error responses should also be verified for proper message and validation details.

3. **Response header validation**

   Check that required headers are returned correctly, including content type, security-related headers, caching directives, and any correlation or trace headers defined by the API standard.

4. **Application state verification**

   Confirm that successful API actions produce the expected backend changes, such as record creation, updates, deletions, approvals, allocations, and reporting updates.

5. **Positive testing**

   Validate the main expected flows for each supported role, including Employee, IT Admin, and approver scenarios, against the acceptance criteria.

6. **Negative testing**

   Verify that the API handles invalid or conflicting inputs gracefully, including missing required fields, invalid IDs, null or empty values, duplicate submissions, and unauthorized access attempts.

7. **Endpoint isolation testing**

Test each endpoint independently to confirm request schema compliance, authentication handling, response correctness, and any direct side effects.

8. **Multi-step workflow testing**

   Validate end-to-end business flows that involve multiple API calls, such as create request, submit, approve or reject, fetch updated status, and verify reporting output.

9. **Authentication and authorization testing**

   Ensure the API enforces the required authentication method and role-based access control, and rejects missing, invalid, expired, or unauthorized tokens.

10. **Protocol validation**

    Confirm that the API behaves correctly over the required protocol, such as HTTP or HTTPS, in line with the specification and environment policy.

10. Validation phase deliverables

The following deliverables will be completed during the Validation Phase of the project.

I. API Test Plan
II. Test Cases & Execution Results
III. Defect Report
IV. Test Closure Report
V. QA Weekly Report

11. Raising a defect (In Excel at SharePoint)

A defect can be a shortcoming, non-conformance to specified requirements, or variation from expected results. The following details are to be provided in an Excel Sheet while creating/logging a defect:

- **Defect # –** Unique identifier assigned to each defect for tracking and reference purposes.
- **Defect Description –** A concise summary describing the defect, issue, or non-conformance identified during testing.
- **Steps to Reproduce –** Detailed step-by-step instructions to replicate the defect in the test environment.
- **Expected Result –** The anticipated correct outcome or behavior as per the project requirements and specification.
- **Actual Result –** The observed incorrect or unexpected behavior/outcome that occurred when the defect was encountered.
- **Assigned To –** The name of the team member (developer/tester) responsible for resolving the defect.
- **Date Created –** The date and time when the defect was logged in the tracking system.
- **Created By –** The name of the QA team member or tester who identified and reported the defect.
- **Defect Found in Sprint –** The sprint number or release version in which the defect was discovered during testing.
- **Severity –** The impact level of the defect on system functionality (Critical, High, Medium, Low).
- **Priority –** The urgency with which the defect must be addressed (Critical, High, Medium, Low).
- **Status –** The current state of the defect (Open, In Progress, Fixed, Closed, On Hold, Reopened).
- **Defect Type –** The category or component affected by the defect (Database, Front End, Back End, UI/UX, Security, API, etc.).

- **Resolution –** Description of the fix applied or root cause analysis completed to resolve the defect.
- **Defect Closed in Sprint –** The sprint number or release version in which the defect was verified as fixed and closed by QA.
- **Screenshots –** Relevant attachments such as screenshots, videos, logs, or any supporting evidence documenting the defect for reference.

Following is the defect severity & priority while creating/logging a defect:

Defect Severity: The degree of impact a defect has on system functionality, data integrity, security, or business operations, used to classify how serious the defect is from a technical and user-impact perspective.

I. Critical: This severity level implies that the process has been completely shut off, and no further action can be taken.
II. Major: This is a significant flaw that causes the system to fail. However, certain parts of the system remain functional.
III. Medium: This flaw results in unfavorable behavior, but the system remains functioning.
IV. Low: This type of flaw won't cause any major breakdown in the system.

Defect Priority: The level of urgency assigned to resolving a defect, used to determine the order and timeline in which the defect should be fixed based on business risk, release commitments, and operational needs.

I. Low: The defect is irritant, but a repair can be done once the more serious defects are fixed.
II. Medium: The defect should be resolved during the normal course of the development, but it can wait until a new version is created.
III. High: The defect must be resolved as soon as possible, as it affects the system severely and cannot be used until it is fixed.

12. Testing phase entry/exit criteria

Phase Entry Criteria:

- Pre-configured Test environments made available for the QA team with the necessary logon permissions.
- Test data made available in the test environments.
- Required access permissions for execution of test scripts and testing the applications.

Phase Exit Criteria:

- The team has executed all the planned tests against the assigned applications.
- The developer has resolved all must-fix bugs.
- The QA team has verified and closed all the open defects in the defect management tools.

13.Validation phase / Test schedule

The following table details the main activities to be carried out during the Validation phase of the Project

Activity	Start Date	End Date
Test Plan creation	31/03/2026	
Test Scenario creation	06/04/2026	
Test Case creation	06/04/2026	
Test Cases execution		

14. QA team responsibilities

The following would be the main responsibilities of the QA Team during Validation phase of the Project:

- API Test Plan creation
- Test Cases creation
- Test Data Creation
- Report testing progress / status
- Report Defects in defect management tool
- Re-verification of reported defects and accordingly changing their status

15. Risks & Contingencies

The following table describes the key risks to the success of this test plan and contingencies to address them:

Risk	Contingencies
Test environment access issues – Blocker issue for the test team, as they won't be able to carry out any testing	QA team will coordinate with the Dev team in order to manage the relevant test environments/ Setup and ensure that the team has proper access.
Authorization issues with applications during testing, due to which testing cannot continue	If the issues cannot be resolved by the QA team, the DEV team will assist in resolving the issues.
Web Service availability	In case of unavailability of web services for the QA team to test the functionality, they will coordinate with the Dev team to ensure the services are running

16. Approvals

I have reviewed and approved this document as the official Test Plan Document for the project, and it accurately reflects the current understanding of the testing objectives. Following approval of this document, changes will be governed by the project's change management process.

Date	Name	Position / Role